

Versi 2.3
Agustus 2026



PEMROGRAMAN FUNGSIONAL

MODUL 1 - PEMROGRAMAN FUNGSIONAL MENGGUNAKAN PYTHON

Penulis:

FERA PUTRI AYU L., S.KOM., M.T.

HERDIANA DWI MAHARANI

NUR INDAH AMELIA AGUSTIN

LABORATORIUM INFORMATIKA
Universitas Muhammadiyah Malang

DAFTAR ISI

DAFTAR ISI.....	2
PENDAHULUAN.....	3
TUJUAN.....	3
TARGET MODUL.....	3
PERSIAPAN.....	3
KATA KUNCI.....	3
1. PARADIGMA PEMROGRAMAN FUNGSIONAL.....	4
1.1. TENTANG PARADIGMA FUNGSIONAL.....	4
1.2. KARAKTERISTIK PEMROGRAMAN FUNGSIONAL.....	4
2. KENAPA MENGGUNAKAN PYTHON ?.....	5
2.1. SINTAKS YANG MUDAH DIBACA DAN DIPELAJARI.....	5
2.2. FLEKSIBEL DAN EKOSISTEM YANG KAYA.....	5
3. LET'S TALK ABOUT PYTHON.....	6
3.1. TIPE DATA DAN VARIABEL DALAM PYTHON.....	6
3.1.1. APA ITU VARIABEL DALAM PYTHON?.....	6
3.1.2. APA ITU TIPE DATA DALAM PYTHON?.....	7
3.1.2.1. TEXT TYPE.....	8
3.1.2.2. NUMERIC TYPE.....	9
3.1.2.3. SEQUENCE TYPE.....	12
3.1.2.4. MAPPING TYPE.....	18
3.1.2.5. BOOLEAN TYPE.....	20
3.1.3. TYPE CASTING IN PYTHON.....	20
3.2. INDENTASI.....	21
3.3. FUNCTION IN PYTHON.....	22
3.4. PERCABANGAN DALAM PYTHON.....	25
3.5. PERULANGAN DALAM PYTHON.....	26
3.5.1. PERULANGAN FOR.....	26
3.5.2. PERULANGAN WHILE.....	28
3.6. FUNGSI REKURSIF.....	28
3.7. EXCEPTION HANDLING DALAM PYTHON.....	29
CODE ANALYSIS.....	31
Verifikasi & Deadline.....	31
ASSIGNMENT.....	31
ASSIGNMENT 1.....	31
ASSIGNMENT 2.....	33
ASSESSMENT RUBRIC.....	35
DETAILED ASSESSMENT RUBRIC.....	35



PENDAHULUAN

TUJUAN

1. Sub-CPMK 1: Mahasiswa mampu menganalisis konsep dasar paradigma pemrograman fungsional dalam komputasi data untuk menghasilkan perangkat lunak berkualitas.

TARGET MODUL

1. Memahami konsep dasar dan prinsip-prinsip paradigma pemrograman fungsional.
2. Menguasai penggunaan variabel dan tipe data dasar (*Text, Sequence, Numeric, Mapping, Boolean*) di Python.
3. Menguasai konsep tipe data sekuens (*sequence data types*) dalam Python.
4. Memahami aturan struktur kode dan indentasi dalam Python.
5. Menguasai kontrol alur program menggunakan percabangan (*branching*), perulangan (*looping*), dan *exception handling*.
6. Menguasai konsep fungsi dan fungsi rekursif dalam paradigma fungsional.

PERSIAPAN

1. Bagi mahasiswa Muslim, disunnahkan membaca doa berikut sebelum memulai praktikum.
رَضِيتُ بِاللَّهِ رَبًّا وَبِالْإِسْلَامِ دِينًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُولًا. رَبِّ زِدْنِي عِلْمًا وَارْزُقْنِي فَهْمًا
2. Laptop atau komputer pribadi
3. Google Collab/VS Code/Jupyter Notebook
4. [Source Code Modul 1](#)
5. Motivasi dan komitmen yang kuat untuk belajar

KATA KUNCI

Paradigma Pemrograman Fungsional, Python, Fungsi, Variabel, Tipe Data, Sequence, Flow Control.



1. PARADIGMA PEMROGRAMAN FUNGSIONAL

Pada semester-semester sebelumnya, kita telah membahas berbagai konsep pemrograman, mulai dari Pemrograman Dasar dan Pemrograman Berorientasi Objek (*Object-Oriented Programming/OOP*) hingga Pemrograman Lanjut dan Struktur Data. Sekarang saatnya kita mendalami **Pemrograman Fungsional** (*Functional Programming/FP*), yaitu cara baru untuk menulis kode yang sepenuhnya berfokus pada fungsi!

Sederhananya, dalam pemrograman fungsional, kita membangun fungsi-fungsi yang tugas utamanya adalah melakukan perhitungan tertentu. Setelah fungsi-fungsi tersebut dibuat, kita dapat memanggil dan menggunakannya kembali kapan pun dan di mana pun saat dibutuhkan.

1.1. TENTANG PARADIGMA FUNGSIONAL

Pemrograman fungsional termasuk dalam payung pemrograman deklaratif (*declarative programming*). Fokus sepenuhnya pada "apa yang harus dicapai", bukan "bagaimana cara mencapainya." Kebalikan dari pemrograman deklaratif adalah pemrograman imperatif (*imperative programming*). Dalam pemrograman imperatif, kita menulis instruksi langkah demi langkah agar komputer tahu persis bagaimana sebuah tugas harus dieksekusi. Ini berarti fokus utamanya adalah pada proses atau "cara melakukannya" (*how-to*). Inilah gaya pemrograman yang persis sama seperti yang telah kamu pelajari pada semester sebelumnya.

Sementara itu, pemrograman deklaratif (termasuk pemrograman fungsional) mengalihkan perhatian pada hasil akhir yang diinginkan tanpa perlu merinci mekanisme prosesnya. Untuk mengilustrasikannya:

- ◆ **Pemrograman imperatif** seperti menulis resep lengkap, langkah demi langkah.
- ◆ **Pemrograman deklaratif** seperti memesan kue kering (*pastry*) di sebuah kafe – kamu cukup menentukan kue apa yang kamu inginkan tanpa perlu tahu cara membuatnya.

Karena sifatnya yang lebih deklaratif, kode yang ditulis dalam pemrograman fungsional biasanya lebih singkat dan lebih mudah dibaca, karena kita tidak perlu mengatur detail teknis yang rumit secara mikro (*micromanage*)

1.2. KARAKTERISTIK PEMROGRAMAN FUNGSIONAL

Program fungsional dapat dianggap sebagai kumpulan fungsi yang menerima masukan (*input*) dan menghasilkan keluaran (*output*) tanpa menyimpan atau memodifikasi data apa pun di luar fungsi tersebut (***stateless***). Ini berarti dengan masukan yang persis sama, fungsi akan selalu menghasilkan keluaran yang persis sama. Fungsi semacam ini disebut fungsi murni (***Pure Functions***). Selain itu, terdapat beberapa karakteristik utama lainnya yang mendefinisikan paradigma ini, antara lain:



- ◆ **Rekursi (Recursion):** Menggunakan fungsi rekursif sebagai pengganti perulangan tradisional (`for` dan `while`).
- ◆ **High-Order Functions:** Fungsi yang dapat menerima fungsi lain sebagai argumen atau mengembalikannya sebagai nilai.
- ◆ **Lambda Functions:** Fungsi anonim (tanpa nama) yang digunakan untuk logika sederhana secara langsung (*on-the-fly*).
- ◆ **Functional Tools:** Memanfaatkan alat pemrosesan bawaan seperti `map()`, `filter()`, dan `reduce()`.

Ini adalah konsep-konsep inti yang nantinya akan kamu kuasai seiring berjalannya modul praktikum pemrograman fungsional ini. Namun, jangan khawatir, kita akan mempelajarinya secara bertahap dan tidak langsung memberikan semuanya sekaligus. Oke, mari kita lanjutkan!

2. KENAPA MENGGUNAKAN PYTHON ?

Meskipun konsep pemrograman fungsional dapat diterapkan pada berbagai bahasa (seperti Haskell, Scala, atau Lisp), kita menggunakan **Python** untuk praktikum ini. Python menjembatani kesenjangan antara berbagai gaya pemrograman dengan sempurna, memungkinkan kita mempelajari paradigma fungsional yang kompleks tanpa harus tersesat dalam aturan bahasa yang terlalu rumit.

2.1. SINTAKS YANG MUDAH DIBACA DAN DIPELAJARI

Python terkenal dengan sintaksnya yang bersih, elegan, dan hampir menyerupai bahasa Inggris.

- ◆ **Keuntungan:** Karena Python menghilangkan elemen kode yang berantakan seperti kurung kurawal `{ }` atau titik koma `;`, kamu tidak perlu membuang waktu untuk bergulat dengan sintaks bahasa.
- ◆ **Dalam Pemrograman Fungsional,** menulis *pure functions* atau persamaan *lambda* menjadi sangat bersih, sehingga kamu bisa berfokus 100% pada logika inti dan transformasi matematis, bukan aturan pengkodean yang rumit.

2.2. FLEKSIBEL DAN EKOSISTEM YANG KAYA

Python adalah bahasa multiparadigma. Bahasa ini tidak membatasi kamu pada satu cara pengkodean saja—melainkan memungkinkan kamu memadukan pemrograman berorientasi objek, imperatif, dan Fungsional secara mulus.

- ◆ **Ekosistem:** Python dilengkapi dengan ekosistem pustaka (*libraries*) yang besar dan siap pakai.



- ◆ **Dalam Pemrosesan & Visualisasi Data:** alih-alih membangun mesin visualisasi dari awal, Python memungkinkan kita langsung menggunakan alat canggih seperti Matplotlib atau Plotly. Ini berarti kita dapat menerapkan logika atau kode transformasi dan langsung melihat data serta tren yang divisualisasikan dalam grafik 2D atau 3D yang jelas hanya dengan beberapa baris kode!

3. LET'S TALK ABOUT PYTHON

Pada bagian ini, kita akan melangkah lebih jauh dari sekadar membahas Python sebagai alat dan langsung mendalami dasar-dasar pemrograman dengan Python. Fokus kita adalah mengenali konsep dasar Python, mulai dari variabel dan tipe data hingga struktur kode sederhana.

Python dikenal karena sintaksnya yang ringkas dan mudah dibaca, sehingga sangat cocok untuk mempelajari pemrograman. Melalui poin-poin berikut, kita akan memahami bagaimana Python mengelola data, menangani percabangan, menjalankan perulangan, dan membuat fungsi.

3.1. TIPE DATA DAN VARIABEL DALAM PYTHON

Sebelum kita beralih ke aspek praktis, mari kita pastikan kita memiliki pemahaman yang sama.

3.1.1. APA ITU VARIABEL DALAM PYTHON?

Bayangkan variabel sebagai selembar *sticky note* atau label nama. Variabel adalah label yang kita tempelkan pada sepotong data agar kita dapat menemukannya dan menggunakannya nanti.

Dalam Python, kamu dapat menempelkan label nama pada berbagai jenis data tanpa harus mendeklarasikan tipenya terlebih dahulu. Ini sangat kontras dengan banyak bahasa pemrograman lain yang mewajibkan kamu menentukan tipe data secara ketat sejak awal (seperti `int`, `string`, atau `float`). Lebih hebatnya lagi, Python bersifat sangat dinamis—Kamu bisa melepas label nama tersebut dari sebuah bilangan bulat (*integer*) dan menempelkannya ke teks kapan pun Kamu mau. Perhatikan baris-baris kode di bawah ini dan jalankan untuk melihat fleksibilitas tersebut secara langsung!

```
#Variable Declaration

#Fill in your identity
name = "Your Name Here"
student_id = "Your NIM"
semester = 4
```

Gambar 3.1. Deklarasi Variabel



Pastikan Kamu telah menjalankan blok kode `#Variable Declaration` sebelum menjalankan sel `#print` di bawah ini!

Tips: Kamu dapat menjalankan kembali sel `#print` setelah bereksperimen dengan sel `#redefine` untuk melihat perbedaannya. Jangan lupa laporkan temuan kamu!

```
# Print Variable Contents

print(f"Name \t: {name} \t| Data Type: {type(name)}")
print(f"ID \t: {student_id} \t| Data Type: {type(student_id)}")
print(f"Semester: {semester} \t| Data Type: {type(semester)}")
```

Gambar 3.2. Print Variable Contents

Exercise 1: The Shape-Shifting Variable

```
# (Your NIM)
#redefine variable

student_id = '123456'
semester = False
```

Gambar 3.3. Exercise 1

Analysis Question:

Berdasarkan hasil eksperimen di atas, jelaskan bagaimana perilaku Python ketika variabel (`student_id` dan `semester`) didefinisikan kembali ke tipe data yang sepenuhnya berbeda. Mengapa, menurut kamu, fleksibilitas ini bisa sangat membantu sekaligus berbahaya?

3.1.2. APA ITU TIPE DATA DALAM PYTHON?

Seperti namanya, tipe data mendefinisikan sifat atau karakteristik suatu data. Setiap data memiliki nilai dan setiap nilai termasuk dalam kategori tertentu. Jika merujuk kembali pada analogi sebelumnya di mana variabel adalah label nama, maka tipe data adalah identitas dari objek yang ditemplei label tersebut. Memahami tipe data merupakan pilar mendasar dalam pemrograman karena membantu kita mengenali jenis data yang sedang dikerjakan, ukurannya, serta fungsi-fungsi yang berkaitan dengannya.

Pemahaman yang kuat tentang tipe data akan membuat proses pengkodean jauh lebih efisien. Tipe data tertentu menentukan nilai apa saja yang dapat kita tetapkan dan,



yang lebih penting, operasi apa saja yang dapat kita lakukan pada data tersebut. Python sendiri memiliki rangkaian tipe data yang cukup unik dibandingkan dengan bahasa pemrograman lain. Beberapa tipe data tersebut meliputi:

Text Type:	<code>str</code>
Numeric Types:	<code>int</code> , <code>float</code> , <code>complex</code>
Sequence Types:	<code>list</code> , <code>tuple</code> , <code>range</code>
Mapping Type:	<code>dict</code>
Set Types:	<code>set</code> , <code>frozenset</code>
Boolean Type:	<code>bool</code>
Binary Types:	<code>bytes</code> , <code>bytearray</code> , <code>memoryview</code>
None Type:	<code>NoneType</code>

Gambar 3.4. Built-in Data Types

Mari kita bedah lebih mendalam beberapa tipe data yang tersedia di Python.

3.1.2.1. TEXT TYPE

a. String

Tidak seperti bahasa pemrograman lain seperti C atau Java, Python tidak memiliki tipe data `char` khusus. Di dalam Python, satu karakter tunggal dapat diperlakukan sebagai *string* dengan panjang satu.

```
letter = 'A'
print(letter)
print(type(letter))
```

```
A
<class 'str'>
```

Gambar 3.5. Tipe Data String

Pada kode di atas, variabel `letter` menampung satu karakter, yaitu `'A'`. Ketika Kamu menjalankan `print(letter)`, keluaran yang ditampilkan adalah huruf A. Selain itu, jika Kamu menggunakan fungsi `type(letter)`, Python akan menghasilkan: `<class 'str'>`.

Hal ini membuktikan secara jelas bahwa nilai `'A'` dalam variabel `letter` diperlakukan sebagai objek dari kelas *string* (`str`), bukan sebagai tipe `char` seperti pada beberapa bahasa pemrograman lainnya. Dengan kata lain, Python memperlakukan semua data sebagai objek, termasuk *string*.



b. String Concatenation

Berikut adalah contoh penggabungan *string* (*string concatenation*), yaitu proses menggabungkan dua *string* berbeda dari variabel terpisah menjadi satu variabel tunggal.

```
# Example of String Concatenation

part1 = "Consistency is the key "
part2 = "to mastering any programming language."

success_quote = part1 + part2
print(success_quote)

... Consistency is the key to mastering any programming language.
```

Gambar 3.6. String Concatenation

3.1.2.2. NUMERIC TYPE

Berikut adalah beberapa contoh tipe data numerik yang terdapat di Python:

a. Int

```
number = 10
print(number)
print(type(number))

10
<class 'int'>
```

Gambar 3.7. Tipe Data Integer

Di dalam Python, tipe data *int* digunakan untuk merepresentasikan bilangan bulat (*integers*), seperti -3, 0, 10, atau 1000. Pada bahasa pemrograman lain seperti C/C++ atau Java, tipe data bilangan bulat biasanya memiliki ukuran tetap dan dikategorikan lebih lanjut menjadi *short*, *int*, atau *long*, di mana setiap jenis memiliki batas nilai tertentu yang dapat disimpan.

Namun, Python menawarkan fleksibilitas yang lebih besar karena tipe data *int* secara otomatis menyesuaikan ukurannya berdasarkan nilai yang ditetapkan. Ini berarti kamu tidak perlu repot mengklasifikasikan suatu angka sebagai *short*, *int*, atau *long*. Python mengelola alokasi memori secara mandiri. Perhatikan kode berikut:



```

▶ # Memory Size Experiment
# (Your NIM)

import sys

data1 = 123456789 #Modify the values
data2 = 1234567890 #Modify the values

print('Value = ', data1)
print('Type = ', type(data1))
print('Size = ', sys.getsizeof(data1), 'bytes\n')

print('Value = ', data2)
print('Type = ', type(data2))
print('Size = ', sys.getsizeof(data2), 'bytes')

... Value = 123456789
Type = <class 'int'>
Size = 28 bytes

Value = 1234567890
Type = <class 'int'>
Size = 32 bytes

```

Gambar 3.8. Eksperimen Ukuran Memori Bilangan Bulat

Semakin besar nilai suatu bilangan bulat, semakin banyak bit (byte) yang dibutuhkan, yang berbanding lurus dengan kebutuhan memori untuk nilainya. Sebagai informasi tambahan, berikut adalah gambaran singkat mengenai ukuran penyimpanan data di dalam Python:

Bytes	type	scaling notes
28	int	+4 bytes about every 30 powers of 2
37	bytes	+1 byte per additional byte
49	str	+1-4 per additional character (depending on max width)
48	tuple	+8 per additional item
64	list	+8 for each additional
224	set	5th increases to 736; 21nd, 2272; 85th, 8416; 341, 32992
240	dict	6th increases to 368; 22nd, 1184; 43rd, 2280; 86th, 4704; 171st, 9320
136	func def	does not include default args and other attrs

Gambar 3.9. Ukuran Penyimpanan Data di dalam Python

Kesimpulannya, tipe data `int` di Python bersifat dinamis dan fleksibel. Kita dapat menyimpan bilangan bulat kecil hingga bilangan yang sangat besar tanpa perlu khawatir harus menentukan ukuran tipe data terlebih dahulu.



✦ Exercise 2a: Investigating Int and Float Memory Size

Ubah nilai-nilai pada sel Eksperimen Ukuran Memori Bilangan Bulat (*Integer Memory Size Experiment*) menggunakan kombinasi angka yang berbeda, lalu jalankan kembali sel tersebut dan amati perbedaan ukuran memori yang dibutuhkan!

b. Float

Di dalam Python, tidak ada tipe data double yang terpisah seperti pada beberapa bahasa pemrograman lain (seperti C atau Java). Sebaliknya, Python menggunakan tipe data float untuk merepresentasikan angka desimal, misalnya 3.14, -0.5, atau 2.0.

```
negative_number = -9.10
scientific_str = "2e3"
is_active = True

print(int(negative_number)) # Convert a negative decimal number float
print(float(scientific_str)) # Convert scientific notation string to
print(float(is_active)) # Convert boolean values to float
```

```
-9
2000.0
1.0
```

Gambar 3.10. Contoh Konversi Tipe Data di Python

Kode di atas menunjukkan bagaimana kita dapat mengonversi tipe data lain menjadi float atau int. Sebagai contoh, angka desimal negatif seperti -9.10 dapat dikonversi menjadi bilangan bulat -9, string notasi ilmiah "2e3" dapat diubah menjadi 2000.0, bahkan nilai boolean seperti True dapat dicetak menjadi 1.0. Proses pengubahan satu tipe data ke tipe data lainnya ini dikenal sebagai *casting* (konversi tipe data).

Apakah tipe data float juga membutuhkan lebih banyak byte seiring bertambah besarnya nilainya, seperti halnya tipe data int? Untuk mengetahuinya, lakukan eksperimen sederhana dengan memodifikasi nilai pada kedua data tersebut.

✦ Exercise 2a: Investigating Int and Float Memory Size

Lanjutkan seperti yang kamu lakukan pada #Exercise 2a: Investigating Int and Float Memory Size, **ubah nilainya dengan tipe data float, dan laporkan temuanmu!**

✎ Analysis Question:

Apakah ukuran byte dari tipe data *float* bertambah besar seiring dengan membesarnya nilai, seperti halnya tipe data *int*? Selain itu, mengapa nilai *int* memiliki ukuran memori yang berbeda-beda?



c. Complex

Python menyediakan tipe data complex untuk merepresentasikan bilangan kompleks. Bilangan kompleks terdiri dari bagian riil (*real part*) dan bagian imajiner (*imaginary part*), yang keduanya direpresentasikan sebagai angka pecahan (*floating-point numbers*). Di dalam Python, kamu dapat mendefinisikan bilangan kompleks menggunakan notasi $a + bj$, di mana a merepresentasikan bagian riil dan b merepresentasikan bagian imajiner.

```
z1 = 2 + 3j
z2 = 1 - 1j
print(z1 + z2)
print(z1 - z2)
print(z1 * z2)
print(z1 / z2)
```

```
(3+2j)
(1+4j)
(5+1j)
(-0.5+2.5j)
```

Gambar 3.11. Tipe Data Complex

Tipe data ini sangat bermanfaat ketika kita perlu menyimpan rumus-rumus aljabar dalam program. Selain itu, Python juga memungkinkan kita melakukan operasi aritmatika secara langsung pada bilangan tersebut.

3.1.2.3. SEQUENCE TYPE

Sekuens adalah kumpulan nilai yang diakses melalui indeks dan memiliki beberapa tipe data utama, seperti *array*. Namun, di Python, tipe data sekuens sedikit berbeda dari *array* pada umumnya. Data sekuens lebih dinamis dibandingkan *array*, kita bisa menambahkan elemen sebanyak yang kita inginkan tanpa batasan ukuran seperti pada *array* yang terbatas karena indeks. Selain itu, sekuens juga dapat menyimpan elemen dengan tipe data yang berbeda. Mari kita pelajari beberapa jenis sekuens berikut ini:

a. List

List dalam Python adalah salah satu tipe data sekuens yang paling fleksibel dan paling sering digunakan. List merupakan kumpulan elemen yang berurutan/terindeks (*ordered*) dan dapat diubah (*mutable*). Karena fleksibilitasnya, list sering digunakan dalam pemrograman sehari-hari. Berikut beberapa karakteristik dari tipe data list:

1. *Mutable* (Dapat Diubah)

List bersifat *mutable*, artinya kita bisa menambah, menghapus, atau mengubah elemen di dalamnya setelah list dibuat. Jalankan sel kode berikut secara berurutan!



```
example_list1 = [1, 2]

# Append operation to add an element to the list
example_list1.append(3)

print(example_list1)
```

[1, 2, 3]

```
# List concatenation (adding a list to another list)
example_list2 = example_list1 + [4, 5, 6, 7]
print(example_list2)
```

[1, 2, 3, 4, 5, 6, 7]

Gambar 3.12. Python List Operations

2. Ordered (Berurutan)

Maksud dari berurutan di sini adalah bahwa akses data dalam *list* akan dimulai secara berurutan sesuai dengan urutan indeksnya (berindeks).

Sebelum menjalankan sel kode di bawah ini, pastikan 2 sel kode sebelumnya sudah di-run!

```
# Access elements sequentially from the front using positive indexing
print(example_list2[0])
print(example_list2[1])

# Access elements sequentially from the back using negative indexing
print(example_list2[-1])
print(example_list2[-2])
```

1
2
7
6

Gambar 3.12. Python List Indexing

3. Heterogen (Berbagai Tipe Data)

List tidak kaku, yang berarti *list* mampu menyimpan nilai (*value*) dengan bermacam-macam tipe data seperti contoh di bawah ini:



```
example_list3 = [1, "hello", 3.14, True, 'a']

print(example_list3)

[1, 'hello', 3.14, True, 'a']
```

Gambar 3.13. Berbagai Tipe Data

4. Memperbolehkan Nilai Ganda (*Allow Duplicates*)

List memperbolehkan data yang memiliki nilai (*value*) yang sama karena *list* diakses dan dibedakan berdasarkan indeksnya

```
example_list4 = ["Python Basics", "Data Science", "Machine Learning",
                 "Python Basics", "Web Development"] # "Python Basics" is duplicated
print(example_list4)

['Python Basics', 'Data Science', 'Machine Learning', 'Python Basics', 'Web Development']
```

Gambar 3.14. Percobaan Nilai Ganda

Masih ada beberapa ciri yang dimiliki oleh *list* seperti *indexing*, [slicing](#), *dynamic size*, dan sebagainya. *List* juga memiliki metode bawaan (*built-in methods*) dari Python yang memudahkan kita mengoperasikan data dalam *list*. Silakan mengunjungi tautan berikut untuk lebih detailnya [documentation link](#).

b. Tuple

Tuple adalah urutan objek dalam Python yang bersifat *immutable* (tidak dapat diubah). Artinya, setelah sebuah *tuple* dibuat, isinya tidak bisa diubah, ditambahkan, atau dihapus. Secara bentuk, *tuple* mirip dengan *list*, hanya saja *tuple* menggunakan kurung `()`, sedangkan *list* menggunakan kurung siku `[]`.

1. *Immutable* (Tidak dapat diubah)

Ketika sebuah *tuple* sudah berhasil dibuat, isinya tidak dapat dimodifikasi. Jika kita mencoba mengubah elemennya, Python akan menampilkan pesan *error*.

Exercise 3: Tuple Syntax Challenge

Jalankan sel-sel di atas dan amati baik-baik keluaran (*output*) dari #Eksperimen a maupun #Eksperimen b!



```
# (Your NIM)

#Experiment a

screen_resolution = (1920, 1080)
screen_resolution[0] = 2560
```

```
# (Your NIM)

#Experiment b

screen_resolution = 1920, 1080
print(screen_resolution)
print(type(screen_resolution))
```

Gambar 3.15. Exercise 3

 Analysis Question:

1. Dapatkah kamu mengidentifikasi mengapa Eksperimen A gagal dijalankan (gagal dijalankan) sementara yang kedua berhasil?
2. Apa yang dapat disimpulkan dari hal tersebut tentang bagaimana Python menangani pembuatan *tuple* dibandingkan modifikasi *tuple*?
2. Heterogen (Banyak Tipe Data)
Tuple dapat berisi berbagai macam tipe data sekaligus, mirip seperti *list*.

```
example_tuple2 = ("abc", 34, True, 40, "Male")
print(example_tuple2)
```

('abc', 34, True, 40, 'Male')

Gambar 3.16. Banyak Tipe Data di Tuple

3. Berurutan (*Ordered*)
Tuple memiliki urutan tetap. Artinya, kita bisa mengakses elemen-elemen di dalamnya menggunakan indeks, baik dari depan (indeks positif) maupun dari belakang (indeks negatif).



```
example_tuple3 = (10, 20, 30, 40, 50)

print(example_tuple3[0])    # First element
print(example_tuple3[1])    # Second element
print(example_tuple3[-1])   # Last element
print(example_tuple3[-5])   # Also the first element
```

```
10
20
50
10
```

Gambar 3.16. Tuple Indexing

Untuk mengetahui lebih lanjut tentang tuple, silakan buka tautan [berikut](#).

c. Range

Range dalam Python adalah tipe data yang digunakan untuk menghasilkan urutan angka. *Range* biasanya digunakan dalam perulangan (*loop*) untuk mengulang sejumlah langkah tertentu. *Range* menghasilkan urutan angka yang teratur sesuai dengan parameter yang diberikan, namun tidak menyimpan semua angka dalam memori, melainkan menghasilkan angka satu per satu saat diperlukan. Ciri-ciri *range* dapat kita simak pada contoh berikut:

1. Immutable

Sama seperti *tuple*, setelah didefinisikan, objek *range* tidak dapat diubah. Kita tidak bisa menambah, menghapus, atau mengubah elemen-elemen dalam *range*.

2. Indexing dan Slicing

Kita bisa mengakses elemen-elemen tertentu dalam *range* menggunakan indeks dan *slicing*, mirip seperti pada *list*. Coba perhatikan dan jalankan kode berikut:

```
my_range = range(1, 15)

print(my_range[3])          # Indexing: Accesses the 4th element in the sequence
print(my_range[3:8])        # Slicing: Accesses a sub-sequence of elements
```

```
4
range(4, 9)
```

Gambar 3.17. Indexing and Slicing

3. Iterasi (Iteration)

Objek *range* dapat diiterasi menggunakan *loop for* yang sangat berguna untuk mengulangi blok kode dengan jumlah langkah yang ditentukan.



```
# range with 1 argument
for i in range(3):
    print(i)
```

0
1
2

```
# range with 2 arguments: first argument = start, second argument = end
for i in range(4, 8): # prints numbers from 4 up to (but not including) 8
    print(i)
```

4
5
6
7

```
# range with 3 arguments: start, end, and step (increment)
for i in range(1, 10, 3):
    print(i)
```

1
4
7

Gambar 3.18. Python `range()` Function Arguments

🔗 Exercise 4: Countdown Range Analysis

Buatlah sekuens hitung mundur menggunakan `range()` (tanpa *if-else*) dengan titik awal yang ditentukan oleh 2 digit terakhir Nomor Induk Mahasiswa (NIM) kamu.

NIM Ganjil: Hitung mundur dari 2 digit terakhir NIM kamu hingga turun menjadi 1, melewati 2 angka sekaligus (dekremen sebesar -2). Simpan dan tampilkan hasilnya sebagai sebuah List.

NIM Genap: Hitung mundur dari 2 digit terakhir NIM kamu hingga 0, melewati 3 angka sekaligus (dekremen sebesar -3). Simpan dan tampilkan hasilnya sebagai Tuple.

📝 Analysis Question:

Jelaskan logika berdasarkan kode yang telah kamu buat.



3.1.2.4. MAPPING TYPE

Dalam Python, kamus atau *dictionary* adalah tipe data yang digunakan untuk menyimpan nilai dalam format pasangan kunci-nilai (*key-value*). *Dictionary* sangat berguna ketika kita ingin mengasosiasikan kunci unik dengan nilai tertentu. Berikut merupakan ciri dari *dictionary*:

- Key (Kunci) tidak boleh duplikat
- *Changeable* (dapat diubah)
- Pengaksesan menggunakan *Key*

Bagaimana cara membuat *dictionary*? Kode berikut adalah contoh deklarasi tipe data *dict*:

```
# Create a Dictionary

student_profile = {
    "name": "SpongeBob SquarePants",
    "student_id": "2409116001",
    "semester": 3
}

print(student_profile)

{'name': 'SpongeBob SquarePants', 'student_id': '2409116001', 'semester': 3}
```

Gambar 3.19. Membuat Dictionary

Berikut adalah beberapa operasi yang dapat dilakukan pada data *dictionary*:

```
# Adding new data
student_profile['job_title'] = "Fry Cook"
student_profile['hobby'] = "Blowing Bubbles"

# Modifying the value of an existing key
student_profile['semester'] = 4

# Accessing a value using its key
print('Character Name:', student_profile['name'])
print(student_profile)

Character Name: SpongeBob SquarePants
{'name': 'SpongeBob SquarePants', 'student_id': '2409116001', 'semester': 4, 'job_title': 'Fry Cook', 'hobby': 'Blowing Bubbles'}
```

Gambar 3.20. Modifikasi Nilai



```
# Developing SpongeBob's profile into a nested dictionary
student_profile = {
    "name": "SpongeBob SquarePants",
    "student_id": "2409116001",
    "semester": 3,
    "account_details": {
        "username": "spongebob_frycook",
        "email": "spongebob@krustykrab.com"
    },
    "emergency_contact": {
        "contact_name": "Patrick Star",
        "relationship": "Best Friend",
        "phone_number": "081234567890"
    }
}

# Accessing a specific value from the nested dictionary
student_email = student_profile["account_details"]["email"]

# Displaying the retrieved nested information
print("Emergency Contact:", student_profile["emergency_contact"]["contact_name"])
print("Student Email:", student_email)
print(student_profile)

Emergency Contact: Patrick Star
Student Email: spongebob@krustykrab.com
{'name': 'SpongeBob SquarePants', 'student_id': '2409116001', 'semester': 3, 'account_details': {'username': 'spongebo
```

Gambar 3.21. Mengakses Nilai dalam *Nested Dictionary*

✦ Exercise 5: Working with Dictionaries

Lengkapi kode di bawah ini dengan mengisi bagian kosong (____) untuk membuat, memperbarui, dan mengelola sebuah *dictionary* dalam Python.

```
# (Your NIM)

# 1. Create a dictionary with your own data
student_info = {
    "name": "____",
    "nim": "____",
    "semester": 4
}

# 2. Add a new key-value pair for your "major"
student_info["____"] = "Informatics"

# 3. Try to update an EXISTING key (change semester to 5)
student_info["semester"] = ____

print("Current Dictionary:", student_info)

# 4. Uncomment the line below to see what happens
# print(student_info["gpa"])
```

Gambar 3.22. Exercise 5



Analysis Question:

1. Ketika kamu memperbarui key "semester" pada langkah ke-3, apakah Python membuat key duplikat atau menimpa nilai yang sudah ada?
2. Apa pesan *error* eksplisit yang diberikan oleh Python ketika kamu *uncomment* dan menjalankan langkah ke-4? Apa arti dari *error* tersebut?

3.1.2.5. BOOLEAN TYPE

Menyatakan nilai benar (True) yang bernilai 1, atau salah (False) yang bernilai 0

```
a = 5 > 6 # false
b = 6 > 5
print(type(a))
print(a, b)
```

```
<class 'bool'>
False True
```

Gambar 3.23. Boolean Type

3.1.3. TYPE CASTING IN PYTHON

Casting berarti mengubah suatu tipe data menjadi tipe lain dengan cara memanggil fungsi konversi bawaan Python, lalu mengirimkan data sebagai parameternya. Dengan *casting*, kita bisa mengubah int ke string, float ke int, *list* ke *tuple*, dan sebagainya. Coba kalian jalankan sel berikut secara berurutan dan amati hasilnya!

```
# Initial values
x = 1
y = 10.657
z = [10, 9, 8, 7]

print("=== Before Casting ===")
print("x:", x, "| Type:", type(x))
print("y:", y, "| Type:", type(y))
print("z:", z, "| Type:", type(z))
```

```
=== Before Casting ===
x: 1 | Type: <class 'int'>
y: 10.657 | Type: <class 'float'>
z: [10, 9, 8, 7] | Type: <class 'list'>
```

Gambar 3.24. Sebelum Casting



```
# Perform casting
x_str = str(x)      # int to string
y_int = int(y)      # float to int (fractional part is discarded)
z_tuple = tuple(z)  # list to tuple

print("=== After Casting ===")
print("x_str:", x_str, "| Type:", type(x_str))
print("y_int:", y_int, "| Type:", type(y_int))
print("z_tuple:", z_tuple, "| Type:", type(z_tuple))

=== After Casting ===
x_str: 1 | Type: <class 'str'>
y_int: 10 | Type: <class 'int'>
z_tuple: (10, 9, 8, 7) | Type: <class 'tuple'>
```

Gambar 3.24. Sesudah Casting

✚ Exercise 6: Data Type Casting

Ubah tipe data NIM kamu dari *string* menjadi *integer* dan *float*, lalu coba konversikan *string* "hello" menjadi *integer* atau *float* untuk melihat dan mencatat pesan *error* eksplisit yang diberikan oleh Python.

```
NIM = "_write your NIM here_"

# write your code here!
```

Gambar 3.25. Exercise 6

📝 Analysis Question:

Berdasarkan eksekusi kode tersebut, mengapa *string* NIM berhasil dikonversi sementara *string* "hello" gagal? Jelaskan mekanisme mendasar dari fungsi `int()` dan `float()` pada Python terkait literal (*literals*).

3.2. INDENTASI

Indentasi dalam konteks pemrograman adalah pengaturan spasi di awal baris kode yang digunakan untuk menunjukkan struktur dan hierarki blok kode (seperti pada awal paragraf). Dalam beberapa bahasa pemrograman, indentasi hanya digunakan untuk meningkatkan keterbacaan kode.

Pada Python, indentasi memiliki peran yang sangat penting karena digunakan untuk mendefinisikan blok kode, seperti di dalam fungsi, *loop*, kondisi, dan lain-lain. Tidak seperti banyak bahasa pemrograman lainnya yang menggunakan tanda kurung kurawal `{ }` untuk menandai blok kode, Python menggunakan indentasi untuk tujuan ini.

Intinya, di dalam Python kita tidak bisa asal menggunakan `tab` atau spasi untuk mengatur baris agar terlihat rapi. Indentasi atau spasi dalam Python digunakan untuk



menentukan *scope* dan jangkauan *code*. Kalian bingung? Coba lakukan eksperimen berikut:

```

# Incorrect Indentation
x = 10

if x > 5:
print("x is greater than 5") # This will trigger an IndentationError

```

File `"/tmp/ipykernel_2088/2990485529.py"`, line 5

```

print("x is greater than 5") # This will trigger an IndentationError
^
IndentationError: expected an indented block after 'if' statement on line 4

```

Langkah berikutnya: [Jelaskan error](#)

```

# Correct Indentation
x = 10

if x > 5:
    print("x is greater than 5") # Indented with 4 spaces

```

x is greater than 5

Gambar 3.26. Indentasi

Biasanya, indentasi ini diperlukan saat ditemukan titik dua (:). Hal ini merupakan pengganti kurung kurawal { } pada bahasa pemrograman umumnya (seperti pada C/Java). Karena di Python, kurung kurawal { } merupakan identitas dari sebuah data *dictionary*, bukan blok kode.

3.3. FUNCTION IN PYTHON

Jika pada paradigma pemrograman sebelumnya (OOP-Java) kalian lebih familiar dengan istilah *method*, maka pada pemrograman fungsional dengan Python kali ini kita akan banyak menggunakan *function* (eh, jangan salah, di Python juga ada *method* kok, cek [di sini](#) untuk tahu perbedaan keduanya).

Python sendiri memiliki banyak *built-in function*, di antaranya yang sering kita gunakan, antara lain `print()` untuk mencetak objek, `len()` berfungsi mengembalikan (*return*) panjang suatu objek, `type()` berfungsi mengembalikan (*return*) tipe data dari suatu variabel, dan masih banyak lagi yang lainnya. Sedangkan untuk membuat suatu *function*, perlu didefinisikan dengan keyword `"def"`, singkatan dari *"define"*.



```
# How to define a function using the 'def' keyword
def firstExample():
    print("Hello World")
```

```
# How to call/execute a function
firstExample()
```

```
Hello World
```

```
# Function with parameters
def codingSession(partner, project):
    print("Amel and " + partner + " are trying to fix bugs in " + project)

codingSession("Rani", "Final Project")
```

```
Amel and Rani are trying to fix bugs in Final Project
```

Gambar 3.27. Definisi dan Parameter Fungsi dalam Python

Fungsi-fungsi di atas adalah deklarasi fungsi pada Python. Meski kita sudah membangun program dalam satuan fungsi-fungsi, bukan berarti kita sudah mengimplementasikan paradigma fungsional. Seperti halnya fungsi-fungsi di atas, fungsi-fungsi tersebut belum memenuhi paradigma fungsional.

Seperti apakah fungsi dalam paradigma fungsional itu? Mari kita cek kembali materi topik 1 tentang definisi fungsi dalam paradigma fungsional:

Program dalam paradigma fungsional dapat dianggap sebagai kumpulan fungsi yang menerima *input* dan menghasilkan *output* tanpa memengaruhi keadaan (*stateless*) serta tidak memiliki efek samping apa pun (*pure function*).

Kita bisa memperbaiki dan menyusun kode dengan pendekatan yang lebih fungsional untuk mencapai manfaat yang ditawarkan oleh paradigma fungsional. Bagaimana caranya? Pastikan fungsi yang kita bangun memenuhi stKamur berikut:

- *Input*: tambahkan parameter sebagai input data yang dibutuhkan oleh fungsi.
- *Output*: tambahkan nilai kembali (*return value*) sebagai hasil fungsi.
- *Pure*: tidak memiliki efek samping, berarti fungsi tidak mengakses atau mengubah variabel global maupun data lain di luar fungsi.

Dengan demikian, kita dapat menghasilkan program yang lebih elegan, mudah dipahami, dan mudah dikelola. Mari kita perbaiki dan susun ulang kode fungsi sebelumnya dengan pendekatan fungsional seperti berikut:



```
# Function declaration using parameters (reusability)
def generateRepoName(developer, project):
    # Adding return as the output
    return "Creating repository: " + developer + "/" + project

# Assign function to a variable: to store the returned value from the function
callback = generateRepoName("Rani", "functional-programming-lab")

print(callback)
```

Creating repository: Rani/functional-programming-lab

Gambar 3.28. Fungsi dengan Pengembalian Nilai

```
# Function declaration as an expression
def multiply(a, b):
    return a * b

result = multiply(9, 14)
print(result)
```

126

Gambar 3.28. Deklarasi Fungsi sebagai Ekspresi

🔧 Exercise 7: Fixing Indentation & Function Errors

Di subbab sebelumnya, kamu telah mempelajari betapa krusialnya indentasi dalam Python. Sekarang, mari uji pengetahuan tersebut! Kode di bawah ini rusak dan menimbulkan *error* karena sintaks fungsi yang tidak tepat serta indentasi yang tidak sesuai. Perbaikilah kode tersebut agar dapat berjalan dengan lancar:

```
# FIX THIS CODE!
calculate_discount(price, discount):
total = price - (price * discount)
return total

result = calculate_discount(100000, 0.1)
print("Final Price:", result)
```

```
# (Your NIM)

# Write your fixed code here
```

Gambar 3.29. Exercise 7



Analysis Question:

1. Pesan *error* eksplisit apa yang ditunjukkan oleh Python sebelum kamu memperbaiki kode tersebut? Dan jelaskan apa yang kamu lakukan untuk memperbaikinya!
2. Mengapa indentasi yang ketat sangat penting saat mendefinisikan isi sebuah fungsi di Python dibandingkan dengan bahasa pemrograman lain?

3.4. PERCABANGAN DALAM PYTHON

Sama seperti bahasa pemrograman lainnya, Python juga memiliki percabangan seperti `if`, `else`, dan `elif`. Namun, Python tidak memiliki percabangan `switch-case`. Penerapan percabangan ini mirip dengan bahasa pemrograman yang telah kalian pelajari. Untuk lebih memahaminya, kalian bisa menjalankan baris program di bawah ini.

```
# if condition

score = 80

if score >= 75:
    print("Congratulations! You passed.")

Congratulations! You passed.
```

```
# if else condition

score = 60

if score >= 75:
    print("Congratulations! You passed.")
else:
    print("Sorry, you did not pass.")

Sorry, you did not pass.
```

```
# if elif else condition

score = 77

if score >= 80:
    print("A")
elif score >= 75 and score < 80:
    print("B")
else:
    print("C")

B
```

Gambar 3.30. Percabangan Dalam Python



3.5. PERULANGAN DALAM PYTHON

Dalam pemrograman, kita sering kali perlu mengulang blok kode tertentu beberapa kali. Daripada menuliskan kode yang sama berulang-ulang, kita menggunakan perulangan (*loop*). Python menyediakan dua jenis perulangan utama: *for loop* (biasanya digunakan ketika jumlah iterasi diketahui) dan *while loop* (digunakan ketika perulangan bergantung pada kondisi tertentu). Mari kita lihat betapa mudahnya Python menangani perulangan!

3.5.1. PERULANGAN FOR

Perulangan *for* pada Python adalah perintah yang digunakan untuk melakukan iterasi pada sebuah *sequence* atau data koleksi, seperti *List*, *Tuple*, *String*, *range*, dan lainnya. *For* pada Python memiliki perilaku yang berbeda dengan *for* pada kebanyakan bahasa pemrograman yang lain, karena pada Python ia sangat berkaitan dengan data *sequence* atau data kolektif. Mungkin kalau dibandingkan dengan bahasa lain, *for* pada Python lebih dikenal sebagai *foreach*. Perhatikan contoh berikut:

```
# for loop on a sequence type (List)

daily_pipeline = ["Write Code", "Merge Conflict", "Emotional Eating", "Pray for the Code", "Sleep"]

# for loop on a list (sequence type)
for activity in daily_pipeline:
    print(activity)
```

Write Code
Merge Conflict
Emotional Eating
Pray for the Code
Sleep

Gambar 3.31. Perulangan For pada Tipe Sekuens

```
# for loop on a string

for x in "Functional":
    print(x)
```

F
u
n
c
t
i
o
n
a
l

Gambar 3.32. Perulangan For pada String



```
# break keyword

for activity in daily_pipeline:
    print(activity)
    if activity == "Merge Conflict":
        print("Panic mode activated! Shutting down everything...")
        break
```

```
Write Code
Merge Conflict
Panic mode activated! Shutting down everything...
```

Gambar 3.33. Perulangan For Dengan Keyword Break

```
# continue keyword

for activity in daily_pipeline:
    if activity == "Merge Conflict":
        print("Skip! Immediate alt+f4, delete git, and find a beach to cry on.")
        continue
    print(activity)
```

```
Write Code
Skip! Immediate alt+f4, delete git, and find a beach to cry on.
Emotional Eating
Pray for the Code
Sleep
```

Gambar 3.34. Perulangan For Dengan Keyword Continue

```
# range in a for loop

for x in range(6):
    print("Astagfirullah")
```

```
Astagfirullah
Astagfirullah
Astagfirullah
Astagfirullah
Astagfirullah
Astagfirullah
```

Gambar 3.35. Perulangan For pada Range



```
# nested for loop

for x in range(3):
    for y in range(5):
        print(y, end=" ") # prevents a new line
    print() # creates a new line
```

```
0 1 2 3 4
0 1 2 3 4
0 1 2 3 4
```

Gambar 3.36. Nested For Loop

3.5.2. PERULANGAN WHILE

Pada Python, *while loop* mirip dengan *while loop* pada bahasa pemrograman lain. Namun, tidak ada *do-while* di Python. Tidak seperti *for loop* yang memiliki aturan khusus, *while loop* di Python cukup mudah dipahami. Kalian bisa mencoba langsung *while loop* di Python dengan menjalankan program di bawah ini.

```
PIN = 1234
enteredPIN = 0

while enteredPIN != PIN:
    enteredPIN = int(input("Enter your PIN: "))
else:
    print("That's correct!")
```

```
Enter your PIN: 1234
That's correct!
```

Gambar 3.37. Perulangan While

3.6. FUNGSI REKURSIF

Fungsi rekursif adalah fungsi yang melakukan perulangan dengan mengacu pada dirinya sendiri. Dalam paradigma pemrograman fungsional, rekursi adalah cara penting untuk melakukan iterasi pada data, menggantikan penggunaan *loop* konvensional.

```
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n - 1)

print(factorial(7))
```

```
5040
```

Gambar 3.37. Recursive Function



Rekursi dalam paradigma fungsional mempromosikan penggunaan fungsi murni dan menghindari perubahan keadaan (*state*) yang tidak perlu. Fungsi murni adalah fungsi yang hasilnya hanya bergantung pada *inputnya* dan tidak mengubah variabel di luar fungsi tersebut. Terkait hal ini, akan kita bahas lebih lanjut pada modul selanjutnya.

🔗 Exercise 8: Analyzing a Recursive Function

Perhatikan fungsi rekursif di bawah ini. Fungsi ini dirancang untuk menghitung jumlah total angka dari *n* turun ke 1 (misalnya, jika *n*=3, fungsi ini menghitung $3+2+1=6$).

```
def total_sum(n):
    print(f"Current n: {n}")
    if n == 1:
        return 1
    else:
        return n + total_sum(n - 1)

# Running the function
result = total_sum(3)
print("Final Result:", result)
```

```
Current n: 3
Current n: 2
Current n: 1
Final Result: 6
```

Gambar 3.38. Exercise 8

📝 Analysis Question:

Jelaskan langkah demi langkah bagaimana `total_sum(3)` dieksekusi, dan jelaskan bagaimana *base case* (kasus dasar) menghentikan fungsi ini agar tidak memanggil dirinya sendiri selamanya!

3.7. EXCEPTION HANDLING DALAM PYTHON

Exception Handling adalah mekanisme yang digunakan untuk mencegah dan menangani *runtime error* (kesalahan saat program berjalan), mencegah program *crash* (berhenti secara paksa), serta memungkinkannya untuk pulih atau berhenti dengan anggun. Python menyediakan 4 blok inti untuk tujuan ini: `try`, `except`, `else`, dan `finally`.

4 Blok Inti & Peran Masing-Masing

- **try:** Blok tempat kamu meletakkan kode yang berpotensi memicu error (misalnya pembagian dengan nol, tipe data yang tidak valid, atau pengindeksan di luar batas).
- **except:** Blok yang dieksekusi hanya jika terjadi error di dalam blok `try`. Kamu dapat menentukan dan menangkap jenis error tertentu di sini.



- **else:** Blok yang dieksekusi hanya jika blok **try** berjalan dengan sukses tanpa menimbulkan *error* apa pun.
- **finally:** Blok yang akan selalu dieksekusi, terlepas dari apakah terjadi *error* atau kode berjalan dengan sukses. Biasanya digunakan untuk operasi pembersihan (*clean-up*).

```
def robust_divide(numerator, denominator):
    try:
        # High-risk zone: could be division by zero OR a data type mix-up
        result = numerator / denominator
    except ZeroDivisionError:
        # Runs ONLY if the denominator is 0
        print("Math Error: Division by zero is not allowed!")
    except TypeError:
        # Runs ONLY if one of the inputs isn't a number (like a string)
        print("Type Error: You can only perform division using actual numbers!")
    else:
        # Runs ONLY if the math was completely successful
        print(f"Calculation Successful! Result: {result}")
    finally:
        # ALWAYS runs, rain or shine
        print("Calculator cleanup: Done.")

print("--- Test 1: Perfect Math ---")
robust_divide(20, 4)

print("\n--- Test 2: Dividing by Zero ---")
robust_divide(20, 0)

print("\n--- Test 3: Math with Text ---")
robust_divide(20, "five")
```

Gambar 3.39. Exception Handling

Fungsi `robust_divide` bertindak sebagai jaring pengaman kode dengan mencoba mengeksekusi operasi pembagian di dalam blok **try**. Jika terjadi *error* matematis akibat pembagian dengan nol, program akan langsung melompat ke blok **except ZeroDivisionError** untuk menampilkan peringatan khusus. Sementara itu, jika masalahnya berasal dari tipe data yang tidak valid seperti teks, hal tersebut akan dicegat oleh blok **except TypeError**.

Ketika perhitungan berjalan dengan sangat lancar tanpa hambatan apa pun, blok **else** akan mengambil alih untuk mencetak hasil yang berhasil. Apa pun skenario yang terjadi—baik kode berhasil tanpa cela maupun menemui hambatan total—blok **finally** di bagian paling akhir dijamin akan selalu berjalan sebagai langkah penutup akhir.



CODE ANALYSIS

Tugas Code Analysis terdiri dari Exercise (1 sampai 8) dan Analysis Questions. Selesaikan exercises pada Notebook, dan tulis jawabanmu di [Template](#) laporan yang disediakan.

Verifikasi & Deadline

- Praktikan **wajib** sudah menyiapkan atau **mengerjakan tugas sebelum pekan materi praktikum** dimulai.
- Praktikan **wajib menunjukkan** pengerjaan Exercise pada Notebook yang sedang berjalan serta laporan praktikum kepada asisten **selama sesi materi berlangsung** untuk mendapatkan penilaian.
- Jika tidak menunjukkan exercise dan laporan kepada asisten, praktikan **tetap wajib mengumpulkan** laporan ke iLab/miLab dalam format **PDF** paling lambat **1 hari setelah praktikum (H+1)**. Namun, **nilai Laporan Praktikum -40 poin dan mendapat nilai 0 untuk Exercise Implementation**.
- Jika terdapat revisi yang diberikan oleh asisten, baik pada Exercise maupun Explanation Report, maka **praktikan wajib menyelesaikan revisi tersebut selama sesi praktikum berlangsung**. **Tidak ada penerimaan hasil revisi pada h+1 atau di iLab**.

ASSIGNMENT

ASSIGNMENT 1

Learning Objective:

Setelah menyelesaikan tugas ini, mahasiswa diharapkan dapat:

1. Memproses dan mengolah data berbasis list menggunakan fungsi dan loop.
2. Menerapkan logika kondisional untuk menyaring data berdasarkan beberapa kriteria.
3. Bekerja dengan struktur data immutable (tuple) dan memverifikasi tipe data.
4. Mengimplementasikan fungsi rekursif untuk menyelesaikan masalah tanpa menggunakan loop atau fungsi bawaan.
5. Menerapkan exception handling untuk memvalidasi data dan menangani error.

Inisialisasi nilai mentah berikut di bagian awal *pipeline* program kamu:

```
# List of Lists containing [Student_Name, Assignment_Score, Exam_Score]
raw_grades = [
    ["Amel", 78, 80],
    ["Dika", 78, 82],
    ["Nisrina", 57, 65],
    ["Nayla", 60, 70],
    ["Fatih", 77, 72],
    ["Rani", 68, 55],
]
```



Langkah-Langkah Eksekusi:

1. Buatlah dua function yang memproses daftar `raw_grades` dan mengembalikan keluaran ganda (*dual output*)

- Function 1

Gunakan perulangan (*loop*) untuk menelusuri setiap data mahasiswa dan menghitung **Nilai Akhir (Final Score)** menggunakan rumus berikut:

$$\text{Final Score} = (\text{Assignment Score} \times 0.4) + (\text{Exam Score} \times 0.6)$$

Tambahkan *Final Score* yang telah dihitung sebagai elemen ke-4 pada daftar masing-masing mahasiswa.

- Function 2

Filter mahasiswa yang Lulus dan Tidak Lulus berdasarkan batasan logika bersyarat

$$\text{Exam Score} \geq 60 \text{ AND Final Score} \geq 70$$

- Return Dual Output

Fungsi harus mengembalikan:

- Function 1: Daftar yang berisi semua mahasiswa dengan data 4 elemen yang telah diperbarui: **[Name, Assignment_Score, Exam_Score, Final_Score]**.
 - Function 2: Daftar yang hanya berisi nama-nama mahasiswa yang lulus dan tidak lulus.
2. Ubah daftar nama mahasiswa yang terfilter, yang lulus (untuk **NIM Ganjil**) dan tidak lulus (untuk **NIM Genap**) dari Pertanyaan 1 menjadi sebuah *Tuple* yang immutable. Cetak *tuple* yang dihasilkan dan tampilkan tipe data yang telah diverifikasi menggunakan fungsi bawaan Python `type()` untuk membuktikan keberhasilan konversi.
 3. Buatlah sebuah fungsi rekursif bernama `count_students_recursive` untuk menghitung berapa banyak mahasiswa yang ada di dalam *tuple* dari Pertanyaan 2. Tangani proses verifikasi nama menggunakan blok try-except-else yang terstruktur dalam sebuah fungsi bernama `verify_student`.
 - Batasan: Kamu dilarang menggunakan *loop for* atau *while*, atau fungsi bawaan `len()`, dalam fungsi rekursif. Kamu harus menggunakan rekursi dengan melacak indeks integer dan mengarahkannya langkah demi langkah menuju *base case*.
 - Exception Handling Requirement: Implementasikan blok *try-except-else* untuk memverifikasi nama mahasiswa. Jika nama yang diperiksa tidak ada di dalam *tuple*, tangkap secara eksplisit `ValueError` di dalam blok *except* untuk menampilkan pesan *error* kustom. Jika nama ditemukan, tangani di dalam blok *else*.



4. Live Modification: Asisten akan memberikan satu tugas modifikasi sederhana. Tugas ini akan mencakup poin-poin konsep berikut:
 - Mengubah aturan logika kondisi di dalam *loop* (untuk memperketat atau mengganti kriteria filter data).
 - Menampilkan hasil urutan data menggunakan metode *slicing* (untuk membatasi atau mengatur bagian data yang ingin ditampilkan).

Silakan buka [MODUL 1-FUNGSI-26.ipynb](#) untuk mendapatkan contoh output code.

ASSIGNMENT 2

Learning Objective:

Setelah menyelesaikan tugas ini, mahasiswa diharapkan dapat:

1. Menerapkan Tuple dan Slicing untuk menampilkan data secara dinamis.
2. Menggunakan Dictionary dan Nested Dictionary untuk autentikasi dan data profil.
3. Menggunakan List of Dictionaries untuk mengelola data dinamis.
4. Merancang fungsi program (registrasi, login, manipulasi data) sesuai dengan struktur data yang tepat.
5. Menerapkan session lock agar data hanya dapat diakses oleh pemiliknya.
6. Menerapkan while loop dan try-except untuk validasi input.

Aturan Penugasan:

Silakan isi tema yang akan kalian gunakan di sini: [link pengumpulan tema](#). GUNAKAN AKUN UMM.

1. **Tema Utama:** Sistem Analitik Game ATAU Pencatatan Data Eksperimental (contoh: Pemain Klub Sepak Bola dan Catatan Suhu Harian).
2. **Yang Tidak Diperbolehkan:**
 - Segala sistem yang bersifat manajemen/transaksional seperti manajemen toko, kasir, perpustakaan, dll.
 - Tema yang dipilih tidak boleh sama dengan contoh yang diberikan atau identik dengan tema milik teman sekelas.
3. **Deadline:** Tema harus dikumpulkan paling lambat 1 hari sebelum sesi demo Modul 1 (H-1). Jika dikumpulkan setelah batas waktu tersebut, nilai Tugas 2 akan dikurangi 10 poin untuk setiap instrumen penilaian.

Tipe Data:

1. Menu: Gunakan *Tuple* untuk menyimpan pilihan navigasi yang bersifat tetap (*fixed*) .
2. Autentikasi & Data Profil: Gunakan *Dictionary* untuk memetakan nama pengguna (*username*) ke kata sandi (*password*), serta *Nested Dictionary* untuk menghubungkan nama pengguna tersebut dengan metadata profil.
3. Main Data: Gunakan *List of Dictionaries* untuk menyimpan catatan dinamis yang berisi bidang data untuk objek tertentu.



Persyaratan Fungsi Program

Tujuan utama dari tugas ini adalah merancang data *mock* (palsu/simulasi) dan membuat fungsi terpisah (penamaan fungsi bebas) untuk masing-masing fitur berikut:

1. **Menu (Tuple & Slicing):** Simpan menu lengkap di dalam sebuah *Tuple*. Buat fungsi untuk menampilkannya secara dinamis menggunakan *Slicing* (membedakan menu untuk tamu vs. menu untuk pengguna yang sudah *login*).
2. **Registrasi (Register):** Buat fungsi untuk menambahkan data pengguna baru secara langsung ke dalam *Account Dictionary* dan *Profile Nested Dictionary*, serta pastikan ID-nya unik dan belum pernah digunakan.
 - **Data Akun:** *Dictionary* (ID sebagai *key*, *Password* sebagai *value*).
 - **Data Profil:** *Nested Dictionary* (ID sebagai *key*, Rincian Profil [nama, nomor telepon, peran, dll.] sebagai *value*).
3. **Login:** Validasi apakah ID dan *Password* yang dimasukkan cocok dengan kredensial pada *Account Dictionary*.
4. **Manipulasi Data:** Buat fungsi terpisah untuk memanipulasi 1 dataset tambahan yang sesuai dengan tema kamu. Jangan gunakan istilah kaku seperti *Create/Read/Update/Delete* (CRUD); gunakan nama yang lebih kontekstual.
 - **Data Tambahan:** Harus berupa *List of Dictionaries* dengan tipe data yang beragam. Sebutkan alasan memilih struktur ini.
 - **Strict Session Lock:** Data harus dikunci per akun. Pengguna hanya dapat mengakses dan memanipulasi data yang terikat langsung pada *key* ID mereka yang sedang *login*.
5. **Input Validation:** Terapkan *loop* (menggunakan *while loop*) dan blok *try-except* secara eksklusif selama proses *input* pengguna untuk mendeteksi dan menangani tipe data yang tidak valid sebelum diproses lebih lanjut.
6. **Persyaratan Data Mock:** Kamu wajib menyiapkan 3 data mock untuk setiap dataset yang dibutuhkan.

Silakan buka [MODUL 1-FUNGSI-26.ipynb](https://github.com/UMM-Informatics-Laboratory/2026-IPYNB/blob/main/1-FUNGSI/26.ipynb) untuk mendapatkan contoh output code.



ASSESSMENT RUBRIC

Assessment Aspect	Points (Total 100%)
Code Analysis	10
Exercise Implementation	5
Practicum Report	5
Assignment 1	45
Code / Feature Completeness	15
Live Modification	15
Understanding	15
Assignment 2	45
Feature Completeness & Flow Logic	10
Core Data Structures & State Management	10
Sequence Manipulation & Validation	10
Understanding	15

DETAILED ASSESSMENT RUBRIC

Code Analysis	Points (Total 10%)
Exercise Implementation	5
Seluruh Exercise selesai dikerjakan dengan benar (poin akumulasi dari seluruh exercise yang ada).	100
Exercise dikerjakan, namun screenshot kode/output salah	-5 (setiap exercise)
Jawaban tidak mengandung identitas praktikan (Nama/NIM)	-2 (setiap exercise)



Selesai melebihi jam praktikum	0 (setiap exercise)
Practicum Report	5
Menggunakan template resmi, seluruh bagian explanation diisi dengan jawaban yang benar dan jelas	100
Jawaban salah atau jawaban kurang tepat	-5 (setiap exercise)
Penjelasan jawaban ditemukan identik dengan praktikan lain	0 (setiap exercise)
Report dikirim setelah praktikum (overtime)	-40 (dari nilai akhir)
Tidak mengumpulkan	0
Assignment 1	Points (Total 45%)
Code / Feature Completeness	15
Ketiga tugas berhasil diimplementasikan dengan benar.	100
Semua tugas berjalan, tetapi terdapat error kecil pada logika dan formatting.	80
Fungsionalitas sebagian atau melanggar batasan operasional dengan menggunakan <code>len()</code> atau loop di dalam fungsi rekursif.	60
Kode gagal dijalankan dan instruksi assignment 1 tidak diikuti.	40
Tidak melakukan apa pun.	0
Live Modification	15
Berhasil menyelesaikan 2 live modifikasi dengan benar.	100
Menyelesaikan modifikasi dengan sedikit kesalahan logika atau bug sintaks.	80
Hanya menyelesaikan sebagian dari tugas yang diberikan.	60
Kode error dan gagal diperbaiki.	40
Tidak melakukan apa pun.	0



Understanding	15
Dapat menjelaskan (4) karakteristik berikut: fungsi Python, struktur data, seleksi kondisi dan perulangan, serta mekanisme rekursi.	100
Dapat menjelaskan sebagian besar konsep (minimal 3), tetapi satu konsep tidak dibahas secara cukup mendalam.	80
Memahami ide umum, tetapi penjelasannya kurang rinci.	60
Tidak dapat menjelaskan sebagian besar kode dan konsep.	40
Tidak dapat menjelaskan konsep sama sekali.	0
Assignment 2	Points (Total 45%)
Feature Completeness & Flow Logic	10
Seluruh fitur berjalan sesuai alur program yang dimaksud, tanpa error.	100
Menyelesaikan seluruh tugas dengan output yang benar, meski memerlukan sedikit penyesuaian eksekusi atau refactoring logika.	80
Fitur berjalan sebagian atau melanggar batasan yang wajib.	60
Hanya memenuhi 1–2 fitur dasar.	40
Instruksi Assignment 2 tidak diikuti.	0
Core Data Structures & State Management	10
Semua struktur data diimplementasikan dengan benar dan hubungan antarstruktur data konsisten.	100
Struktur data sesuai dengan tipe yang diminta, tetapi terdapat kesalahan kecil dalam implementasi.	80
Beberapa struktur data tidak sesuai dengan tipe yang diminta, atau hubungan antarstruktur data tidak konsisten.	60
Struktur data dibuat, tetapi terlalu sederhana dan tidak mencerminkan tema yang dipilih.	40
Tidak ada struktur data yang sesuai dengan spesifikasi yang diminta.	0
Sequence Manipulation & Validation	10
Slicing pada Menu dan while-loop dengan try-except pada semua prompt input diimplementasikan dengan benar dan konsisten.	100



Slicing dan loop validasi input telah diimplementasikan, tetapi terdapat bug kecil.	80
Hanya salah satu dari keduanya yang diimplementasikan dengan benar (menu slicing atau while-loop/try-except validasi input).	60
Gagal memanipulasi data sequence dan melakukan validasi dengan benar.	40
Tidak ada implementasi slicing atau validasi input sama sekali.	0
Understanding	15
Dapat menjelaskan: (1) seluruh alur program, (2) tujuan setiap struktur data, dan (3) alasan pemilihan struktur data untuk data tambahan.	100
Dapat menjelaskan sebagian besar konsep, tetapi satu konsep tidak dibahas secara cukup mendalam.	80
Hanya mampu menjelaskan sebagian konsep tersebut.	60
Tidak dapat menjelaskan sebagian besar konsep.	40
Sama sekali tidak dapat menjelaskan konsep.	0

